Universidade Federal de Ouro Preto - UFOP

Instituto de Ciências Exatas e Biológicas - ICEB

Departamento de Computação - DECOM

Ciência da Computação

Trabalho Prático I: Pesquisa Externa

BCC203 - Estrutura de Dados II

Davi Samuel Machado Soares, Luisa Rafaela Silva Reis, Roberta Teixeira do Nascimento e Luiza Oliveira Fonseca

Professor: Guilherme Tavares de Assis

Ouro Preto
14 de outubro de 2026

Sumário

1 Introdução **1**
1.1 Especificações do problema . . . 1
1.2 Considerações iniciais . . . 1
1.3 Ferramentas utilizadas . . . 1
1.4 Especificações da máquina . . . 1
1.5 Instruções de compilação e execução . . . 2

2 Desenvolvimento **3**
2.1 Contagem das métricas . . . 3
2.2 Método 1 – Acesso sequencial indexado . . . 4
2.3 Método 2 – Árvore binária de pesquisa em memória externa . . . 5
2.4 Método 3 – Árvore B . . . 6
2.5 Método 4 – Árvore B* . . . 8

3 Experimentos **10**
3.1 Limitações da bateria de testes . . . 10

4 Resultados **11**
4.1 Método 1 – Acesso sequencial indexado . . . 11
4.2 Método 2 – Árvore binária de pesquisa em memória externa . . . 11
4.3 Método 3 – Árvore B . . . 12
4.4 Método 4 – Árvore B* . . . 13
4.5 Análise comparativa . . . 14

5 Considerações Finais **16**

Lista de Figuras

1 Comparações médias por pesquisa. . . . 14
2 Tempo de criação do índice. . . . 14

Lista de Tabelas

1 Organização dos arquivos do projeto. . . . 3
2 Combinações não executadas. . . . 10
3 Método 1, situação ascendente. . . . 11
4 Método 2. Situações 1 e 2 com 200.000 e 2.000.000 não foram executadas (Tabela 2). . . . 11
5 Método 3 (ordem 100). Transferências na pesquisa: 0 em todas as linhas. O caso de 2.000.000 não foi executado (Tabela 2). . . . 12
6 Método 4 (páginas de 100 itens). Transferências na pesquisa: 0 em todas as linhas. O caso de 2.000.000 não foi executado (Tabela 2). . . . 13
7 Resumo comparativo dos métodos. . . . 15

Lista de Códigos Fonte

1 Definição do registro (registro.h). . . . 1
2 Estrutura de métricas e macro de contagem (metricas.h). . . . 3
3 Medição das duas fases (main.c). . . . 3
4 Criação da tabela de índice (asi.c). . . . 4
5 Pesquisa no acesso sequencial indexado (asi.c). . . . 4
6 Nodo da árvore binária externa (pB.h). . . . 5
7 Inserção de um nodo à esquerda (trecho de construirArvoreBinaria, pB.c). O caso da direita é simétrico. . . . 5
8 Pesquisa na árvore binária externa (pB.c). . . . 5

9 Página da árvore B (avB.h). 6
10 Inserção recursiva com divisão de página (avB.c). 6
11 Pesquisa na árvore B com busca binária na página (avB.c). 7
12 Página da árvore B* (aE.h). 8
13 Divisão de página interna cheia (trecho de inserirRecursivo, aE.c). 8
14 Pesquisa na árvore B* (aE.c). 9
15 Escolha das 20 chaves do experimento (script de shell). 10

1 Introdução

Este trabalho compara o desempenho de quatro métodos de pesquisa externa: (1) acesso sequencial indexado, (2) árvore binária de pesquisa em memória externa, (3) árvore B e (4) árvore B*. Para cada método são medidos, separadamente para a *criação do índice* e para a *pesquisa*, o número de transferências de itens da memória externa para a interna, o número de comparações entre chaves e o tempo de execução.

Para este trabalho é necessário entregar o código em C e um relatório referente ao que foi desenvolvido. Os algoritmos desenvolvidos são os quatro métodos de pesquisa externa citados acima, baseados em [5]. As árvores B e B* seguem a definição original de [1], revisada em [2], e as análises de custo usam como referência [4] e [3].

A codificação foi feita em C, usando somente a biblioteca padrão e funções POSIX (`clock_gettime`, `fsync` e `posix_fadvise`), sem bibliotecas adicionais.

1.1 Especificações do problema

Os registros ficam em um arquivo binário e cada um tem três campos: `chave` (`int`), `dado1` (`long`) e `dado2` (`char[5000]`), o que dá 5.016 bytes por registro (Código 1).

```c
typedef struct {
  int chave;
  long dado1;
  char dado2[5000];
} TipoRegistro;
```

Código 1: Definição do registro (registro.h).

O programa trabalha com arquivos de 100, 200, 2.000, 20.000, 200.000 e 2.000.000 de registros em três situações de ordem (ascendente, descendente e aleatória), construir o índice do método escolhido e pesquisar uma chave. A saída informa se a chave foi encontrada, mostra o registro completo e as três métricas de criação e de pesquisa.

1.2 Considerações iniciais

Algumas ferramentas foram utilizadas durante a criação deste projeto:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code.
- Linguagem utilizada: C.
- Compilador: `gcc`, automatizado por `Makefile`.
- Ambiente de desenvolvimento da documentação: Overleaf LaTeX. [1]

1.3 Ferramentas utilizadas

Algumas ferramentas foram utilizadas para testar a implementação, como:

- Script de shell que executa o programa para todas as combinações de método, quantidade e situação, uma vez para cada uma das 20 chaves pesquisadas, e calcula as médias.
- */usr/bin/time -l*: medição do pico de memória (RAM) usado pelo processo.

1.4 Especificações da máquina

A máquina onde o desenvolvimento e os testes foram realizados possui a seguinte configuração:

- Processador: Apple M4 (10 núcleos).
- Memória RAM: 16 GB.
- Sistema Operacional: macOS.

[1] Disponível em https://www.overleaf.com/

1.5 Instruções de compilação e execução

Para a compilação do projeto, basta digitar:

Compilando o projeto

```
make TARGET=pesquisa
```

O `Makefile` compila cada módulo (`main.c`, `metricas.c`, `asi.c`, `gerador.c`, `pB.c`, `avB.c` e `aE.c`) com as seguintes opções:

- *-std=c11*: padrão da linguagem C usado no projeto.

- *-Wall -Wextra*: para mostrar todos os possíveis *warnings* do código.

- *-O2*: otimização do código gerado, já que o objetivo é medir desempenho.

Para a execução do programa basta digitar:

Executando o programa

```
./pesquisa <metodo> <quantidade> <situacao> <chave> [-P]
```

Onde:

- `metodo`: 1 (acesso sequencial indexado), 2 (árvore binária), 3 (árvore B) ou 4 (árvore B*);

- `quantidade`: número de registros do arquivo;

- `situacao`: 1 (ascendente), 2 (descendente) ou 3 (aleatória);

- `chave`: chave a ser pesquisada;

- `-P` (opcional): imprime as chaves do arquivo antes da pesquisa, fora de qualquer medição.

Argumentos inválidos, método 1 com arquivo não ascendente, falta de memória e erros de arquivo geram mensagens de erro específicas.

2 Desenvolvimento

O projeto foi organizado em um diretório por método, com um módulo de métricas compartilhado (Tabela 1).

Arquivo	Responsabilidade
`main.c`	Linha de comando, criação do índice, pesquisa e medições
`metricas.c/h`	Contadores de transferências e comparações, relógio
`acesso sequencial indexado/asi.c`	Método 1
`pesquisa binaria/pB.c`	Método 2
`arvore B/avB.c`	Método 3
`arvore Estrela/aE.c`	Método 4

Tabela 1: Organização dos arquivos do projeto.

2.1 Contagem das métricas

- **Transferências**: cada registro ou nodo lido da memória externa com `fread` incrementa um contador global (`metricas.transferencias`).
- **Comparações**: toda comparação entre chaves passa pela macro `COMP`, que incrementa o contador e avalia a expressão. Assim a contagem é uniforme entre os métodos.
- **Tempo**: `clock_gettime(CLOCK_MONOTONIC)`, com resolução de nanossegundos. A impressão de resultados não entra no tempo.

```c
typedef struct {
  long transferencias; /* registros/nodos lidos da memoria externa */
  long comparacoes;    /* comparacoes entre chaves de pesquisa */
  double tempo;        /* segundos */
} Metricas;

extern Metricas metricas;

/* avalia a expressao de comparacao de chaves contando-a */
#define COMP(expr) (metricas.comparacoes++, (expr))
```

Código 2: Estrutura de métricas e macro de contagem (metricas.h).

O `main.c` zera os contadores antes de cada fase, cronometra a função inteira de criação do índice (`criarIndice`) e depois a pesquisa (`buscar`), guardando as métricas de cada fase separadamente (Código 3).

```c
  /* ---- CRIACAO DO INDICE (cronometrada) ---- */
  metricas = (Metricas){0, 0, 0.0};
  double inicio = agora();
  int ok = criarIndice(ctx, nomeArquivo, nomeArvore);
  double fim = agora();
  criacao = metricas;
  criacao.tempo = fim - inicio;

  /* ---- PESQUISA (cronometrada) ---- */
  metricas = (Metricas){0, 0, 0.0};
  inicio = agora();
  int achou = buscar(ctx, chave, &registro);
  fim = agora();
  pesquisa = metricas;
  pesquisa.tempo = fim - inicio;
```

Código 3: Medição das duas fases (main.c).

2.2 Método 1 – Acesso sequencial indexado

O arquivo é dividido em páginas de 100 registros (`ITENS_POR_PAGINA`); a tabela de índice guarda, para cada página, seu número e sua primeira chave (até 20.000 páginas, ou seja, 2.000.000 de registros).

- **Criação**: lê o primeiro registro de cada página e salta os outros 99 com `fseek` (Código 4).
- **Pesquisa**: percorre a tabela sequencialmente até achar a página candidata, lê a página inteira (a última pode estar incompleta) e faz pesquisa sequencial nela (Código 5).
- Só se aplica ao arquivo ascendente; nas situações 2 e 3 o programa recusa a execução com mensagem de erro.

```c
int criarIndiceASI(FILE *arquivo, TipoIndice indice[], int *quantidadePaginas)
    {
  TipoRegistro registro;
  *quantidadePaginas = 0;
  while (fread(&registro, sizeof(registro), 1, arquivo) == 1) {
    metricas.transferencias++; /* 1 registro lido (o 1o de cada pagina) */
    if (*quantidadePaginas >= MAX_PAGINAS) {
      printf("Erro: arquivo possui mais paginas do que o suportado "
             "(MAX_PAGINAS = %d)\n", MAX_PAGINAS);
      return 0;
    }
    indice[*quantidadePaginas].primeiraChave = registro.chave;
    indice[*quantidadePaginas].numeroPagina = *quantidadePaginas + 1;
    (*quantidadePaginas)++;
    /* pula o restante da pagina */
    fseek(arquivo, sizeof(registro) * (ITENS_POR_PAGINA - 1), SEEK_CUR);
  }
  return 1;
}
```

Código 4: Criação da tabela de índice (asi.c).

```c
int pesquisarRegistro(TipoIndice indice[], int quantidadePaginas,
                      TipoRegistro *registro, FILE *arquivo) {
  TipoRegistro pagina[ITENS_POR_PAGINA];
  int paginaAlvo, itensNaPagina;
  long deslocamento;
  /* procura pela pagina onde o registro pode se encontrar */
  paginaAlvo = 0;
  while (paginaAlvo < quantidadePaginas &&
         COMP(indice[paginaAlvo].primeiraChave <= registro->chave))
    paginaAlvo++;
  if (paginaAlvo == 0)
    return 0;
  else { /* a ultima pagina pode nao estar completa */
    if (paginaAlvo < quantidadePaginas)
      itensNaPagina = ITENS_POR_PAGINA;
    else {
      fseek(arquivo, 0, SEEK_END);
      int itensRestantes =
          (ftell(arquivo) / sizeof(TipoRegistro)) % ITENS_POR_PAGINA;
      itensNaPagina = itensRestantes == 0 ? ITENS_POR_PAGINA : itensRestantes;
    }
    /* le a pagina desejada do arquivo */
    deslocamento = (indice[paginaAlvo - 1].numeroPagina - 1) *
                   ITENS_POR_PAGINA * sizeof(TipoRegistro);
    fseek(arquivo, deslocamento, SEEK_SET);
    size_t lidos = fread(&pagina, sizeof(TipoRegistro), itensNaPagina, arquivo
        );
```

```
    metricas.transferencias += (long)lidos;
    /* pesquisa sequencial na pagina lida */
    for (size_t i = 0; i < lidos; i++)
      if (COMP(pagina[i].chave == registro->chave)) {
        *registro = pagina[i];
        return 1;
      }
    return 0;
  }
}
```

Código 5: Pesquisa no acesso sequencial indexado (asi.c).

2.3 Método 2 – Árvore binária de pesquisa em memória externa

A árvore fica inteira em disco, no arquivo `arvorebinaria_q_s.bin`. Cada nodo guarda o registro completo e as posições (índice no arquivo) dos filhos esquerdo e direito, com -1 indicando filho vazio (Código 6).

```
typedef struct {
    long esquerda;
    TipoRegistro registro;
    long direita;
} Nodo;
```

Código 6: Nodo da árvore binária externa (pB.h).

- **Criação**: cada registro do arquivo de dados é lido (1 transferência) e inserido descendo a partir da raiz; cada nodo visitado é lido do arquivo da árvore (1 transferência). O novo nodo é gravado no fim do arquivo e o ponteiro do pai é atualizado no lugar (Código 7).
- **Pesquisa**: desce a partir da raiz lendo um nodo por vez do arquivo, até achar a chave ou chegar a um filho vazio (Código 8).
- É o único método em que tanto a criação quanto a pesquisa usam somente memória externa.

```
  fseek(arquivoArvore, posAtual * sizeof(Nodo), SEEK_SET);
  fread(&nodoAtual, sizeof(Nodo), 1, arquivoArvore);
  metricas.transferencias++; /* nodo lido da arvore (memoria externa) */

  if (COMP(registroLido.chave < nodoAtual.registro.chave)) {
    if (nodoAtual.esquerda == -1) {
      /* atualiza o ponteiro esquerdo do pai */
      nodoAtual.esquerda = posNovoNodo;
      fseek(arquivoArvore, posAtual * sizeof(Nodo), SEEK_SET);
      fwrite(&nodoAtual, sizeof(Nodo), 1, arquivoArvore);
      /* grava o novo nodo no final do arquivo */
      fseek(arquivoArvore, 0, SEEK_END);
      fwrite(&novoNodo, sizeof(Nodo), 1, arquivoArvore);
      inserido = 1;
      totalNodos++;
    } else {
      posAtual = nodoAtual.esquerda;
    }
  }
```

Código 7: Inserção de um nodo à esquerda (trecho de construirArvoreBinaria, pB.c). O caso da direita é simétrico.

```
int pesquisarNaArvoreBinaria(FILE *arquivoArvore, int chaveBuscada,
                             TipoRegistro *registro) {
  long posAtual = 0; /* comeca da raiz */
```

```
  Nodo nodoAtual;
  while (posAtual != -1) {
    fseek(arquivoArvore, posAtual * sizeof(Nodo), SEEK_SET);
    if (fread(&nodoAtual, sizeof(Nodo), 1, arquivoArvore) != 1)
      return 0;
    metricas.transferencias++;
    if (COMP(chaveBuscada == nodoAtual.registro.chave)) {
      *registro = nodoAtual.registro;
      return 1;
    } else if (COMP(chaveBuscada < nodoAtual.registro.chave)) {
      posAtual = nodoAtual.esquerda;
    } else {
      posAtual = nodoAtual.direita;
    }
  }
  return 0;
}
```

Código 8: Pesquisa na árvore binária externa (pB.c).

2.4 Método 3 – Árvore B

Árvore B de ordem 100 (`ORDEM`), com até 200 registros e 201 filhos por página, mantida inteiramente em memória principal (Código 9). Pela definição de [1], toda página exceto a raiz tem entre `ORDEM` e 2·`ORDEM` registros, o que garante altura máxima de $\log_{\texttt{ORDEM}+1}((n+1)/2)$ [3].

- **Criação**: cada registro do arquivo é lido uma vez (1 transferência) e inserido recursivamente; a busca da posição na página é sequencial e, em caso de overflow, a página é dividida e o registro do meio sobe para o pai. A árvore cresce pela raiz (Código 10).
- **Pesquisa**: desce a partir da raiz fazendo pesquisa binária dentro de cada página (Código 11).
- Como as páginas guardam os registros completos, a pesquisa não lê nada do disco: as transferências de pesquisa são 0.

```
#define ORDEM 100
#define MAX_REGISTROS (2 * ORDEM)

typedef struct TipoPagina *TipoApontador;
typedef struct TipoPagina {
  short quantidade;
  TipoRegistro registros[MAX_REGISTROS];
  TipoApontador filhos[MAX_REGISTROS + 1];
} TipoPagina;
```

Código 9: Página da árvore B (avB.h).

```
void inserirRecursivo(TipoRegistro registro, TipoApontador pagina,
                      short *cresceu, TipoRegistro *registroPromovido,
                      TipoApontador *filhoPromovido) {
  long posicao = 1;
  long j;
  TipoApontador novaPagina;
  //caso base: chegou no fundo da arvore
  if (pagina == NULL) {
    *cresceu = TRUE;
    (*registroPromovido) = registro;
    (*filhoPromovido) = NULL;
    return;
  }
  //procura por qual dos filhos descer na arvore
```

```
  while (posicao < pagina->quantidade &&
         COMP(registro.chave > pagina->registros[posicao - 1].chave))
    posicao++;
  //impede que chaves duplicadas sejam inseridas
  if (COMP(registro.chave == pagina->registros[posicao - 1].chave)) {
    printf("Erro: Registro ja esta presente\n");
    *cresceu = FALSE;
    return;
  }
  if (COMP(registro.chave < pagina->registros[posicao - 1].chave))
    posicao--;
  inserirRecursivo(registro, pagina->filhos[posicao], cresceu,
                   registroPromovido, filhoPromovido);
  if (!*cresceu)
    return;
  //se a pagina atual tem espaco, apenas insere a chave e encerra
  if (pagina->quantidade < MAX_REGISTROS) {
    inserirNaPagina(pagina, *registroPromovido, *filhoPromovido);
    *cresceu = FALSE;
    return;
  }
  //overflow: a pagina tem que ser dividida
  novaPagina = (TipoApontador)malloc(sizeof(TipoPagina));
  novaPagina->quantidade = 0;
  novaPagina->filhos[0] = NULL;
  //decide se a chave fica na pagina antiga ou na nova
  if (posicao < ORDEM + 1) {
    inserirNaPagina(novaPagina, pagina->registros[MAX_REGISTROS - 1],
                    pagina->filhos[MAX_REGISTROS]);
    pagina->quantidade--;
    inserirNaPagina(pagina, *registroPromovido, *filhoPromovido);
  } else
    inserirNaPagina(novaPagina, *registroPromovido, *filhoPromovido);
  //transfere a segunda metade da pagina antiga para a nova
  for (j = ORDEM + 2; j <= MAX_REGISTROS; j++)
    inserirNaPagina(novaPagina, pagina->registros[j - 1], pagina->filhos[j]);
  pagina->quantidade = ORDEM;
  novaPagina->filhos[0] = pagina->filhos[ORDEM + 1];
  //o elemento do meio sobe para o pai
  *registroPromovido = pagina->registros[ORDEM];
  *filhoPromovido = novaPagina;
}
```

Código 10: Inserção recursiva com divisão de página (avB.c).

```
int pesquisarNaArvoreB(TipoRegistro *registro, TipoApontador pagina) {
  while (pagina != NULL) {
    int esq = 0, dir = pagina->quantidade - 1, meio;
    while (esq <= dir) {
      meio = esq + (dir - esq) / 2;
      if (COMP(registro->chave == pagina->registros[meio].chave)) {
        *registro = pagina->registros[meio];
        return 1;
      }
      if (COMP(registro->chave < pagina->registros[meio].chave))
        dir = meio - 1;
      else
        esq = meio + 1;
    }
    //se nao encontrou, esq aponta para o ponteiro do filho correto
    pagina = pagina->filhos[esq];
  }
```

```
  return 0;
}
```

Código 11: Pesquisa na árvore B com busca binária na página (avB.c).

2.5 Método 4 – Árvore B*

Os registros ficam apenas nas folhas (até 100 por folha, `MAX_REGISTROS_FOLHA`), encadeadas pelo ponteiro `proximo`; as páginas internas guardam só chaves (até 100, `MAX_CHAVES_INTERNA`) e 101 filhos. A árvore inteira fica em memória principal. Essa organização, com os dados só nas folhas e as páginas internas funcionando como índice, é a variante descrita por [2] e [4], chamada de árvore B* em [5].

A página é um registro variante (`union`), mostrado no Código 12. As páginas internas são alocadas só com o tamanho da parte interna (cerca de 1,2 KB, em vez dos 490 KB de uma folha), o que reduz bastante o uso de memória.

```
typedef enum { PaginaInterna, PaginaFolha } TipoCategoriaPagina;

typedef struct TipoPaginaEstrela *TipoApontadorEstrela;
typedef struct TipoPaginaEstrela {
  TipoCategoriaPagina tipo;
  union {
    struct {
      int quantidadeChaves;
      int chaves[MAX_CHAVES_INTERNA];
      TipoApontadorEstrela filhos[MAX_CHAVES_INTERNA + 1];
    } interna;
    struct {
      int quantidadeRegistros;
      TipoRegistro registros[MAX_REGISTROS_FOLHA];
      TipoApontadorEstrela proximo; //encadeamento das folhas
    } folha;
  } conteudo;
} TipoPaginaEstrela;
```

Código 12: Página da árvore B* (aE.h).

- **Criação**: cada registro do arquivo é lido uma vez e inserido na folha correta. Uma folha cheia é dividida ao meio e a primeira chave da nova folha sobe como separadora. Uma página interna cheia junta as 101 chaves num vetor temporário, divide ao meio e promove a chave central (Código 13).
- **Pesquisa**: pesquisa sequencial nas páginas internas até chegar a uma folha e pesquisa sequencial na folha (Código 14). Assim como na árvore B, não há leitura de disco na pesquisa.

```
    int chaves[MAX_CHAVES_INTERNA + 1];
    TipoApontadorEstrela filhos[MAX_CHAVES_INTERNA + 2];
    int total = MAX_CHAVES_INTERNA + 1;
    filhos[0] = pagina->conteudo.interna.filhos[0];
    for (j = 0; j < MAX_CHAVES_INTERNA; j++) {
      chaves[j] = pagina->conteudo.interna.chaves[j];
      filhos[j + 1] = pagina->conteudo.interna.filhos[j + 1];
    }
    j = MAX_CHAVES_INTERNA;
    while (j > 0 && COMP(*chavePromovida < chaves[j - 1])) {
      chaves[j] = chaves[j - 1];
      filhos[j + 1] = filhos[j];
      j--;
    }
    chaves[j] = *chavePromovida;
    filhos[j + 1] = *filhoPromovido;

```

```
    int meio = total / 2;
    novaPagina = malloc(offsetof(TipoPaginaEstrela, conteudo) +
                 sizeof(((TipoPaginaEstrela *)0)->conteudo.interna));
    novaPagina->tipo = PaginaInterna;

    pagina->conteudo.interna.quantidadeChaves = meio;
    for (j = 0; j < meio; j++) {
      pagina->conteudo.interna.chaves[j] = chaves[j];
      pagina->conteudo.interna.filhos[j + 1] = filhos[j + 1];
    }
    novaPagina->conteudo.interna.filhos[0] = filhos[meio + 1];
    for (j = meio + 1; j < total; j++) {
      novaPagina->conteudo.interna.chaves[j - meio - 1] = chaves[j];
      novaPagina->conteudo.interna.filhos[j - meio] = filhos[j + 1];
    }
    novaPagina->conteudo.interna.quantidadeChaves = total - meio - 1;

    *chavePromovida = chaves[meio];
    *filhoPromovido = novaPagina;
    *cresceu = true;
```

Código 13: Divisão de página interna cheia (trecho de inserirRecursivo, aE.c).

```
int pesquisarNaArvoreBEstrela(TipoRegistro *registro,
                              TipoApontadorEstrela pagina) {
  int posicao;
  if (pagina == NULL)
    return 0;
  if (pagina->tipo == PaginaInterna) {
    posicao = 1;
    while (posicao < pagina->conteudo.interna.quantidadeChaves &&
           COMP(registro->chave > pagina->conteudo.interna.chaves[posicao -
               1]))
      posicao++;
    if (COMP(registro->chave < pagina->conteudo.interna.chaves[posicao - 1]))
      return pesquisarNaArvoreBEstrela(
          registro, pagina->conteudo.interna.filhos[posicao - 1]);
    return pesquisarNaArvoreBEstrela(registro,
                                     pagina->conteudo.interna.filhos[posicao])
                                         ;
  }
  posicao = 1;
  while (posicao < pagina->conteudo.folha.quantidadeRegistros &&
         COMP(registro->chave > pagina->conteudo.folha.registros[posicao - 1].
             chave))
    posicao++;
  if (pagina->conteudo.folha.quantidadeRegistros > 0 &&
      COMP(registro->chave == pagina->conteudo.folha.registros[posicao - 1].
          chave)) {
    *registro = pagina->conteudo.folha.registros[posicao - 1];
    return 1;
  }
  return 0;
}
```

Código 14: Pesquisa na árvore B* (aE.c).

3 Experimentos

Foram medidos, para a criação do índice e para a pesquisa, o número de transferências, o número de comparações entre chaves e o tempo de execução, na máquina descrita na Seção 1.4 (Apple M4, 16 GB de RAM, macOS, `gcc -O2`). Também foi medido o pico de memória do processo com `/usr/bin/time -l`.

- Quantidades: 100, 200, 2.000, 20.000, 200.000 e 2.000.000 registros, nas três situações de ordem.
- Em cada combinação foram pesquisadas 20 chaves distintas, igualmente espaçadas entre 1 e n (todas existentes), conforme o Código 15. As tabelas mostram as médias dessas 20 pesquisas; todas as chaves foram encontradas em todos os testes.
- Métodos 1 e 2: foram medidos com uma versão anterior do programa, que tinha um modo de experimento em que o índice era criado uma única vez e as 20 chaves eram pesquisadas em seguida. Como o código desses dois métodos não mudou depois, essas medições foram mantidas.
- Métodos 3 e 4: foram medidos novamente depois que o tamanho das páginas passou para 100 (árvore B de ordem 100 e árvore B* com 100 itens por página). Como o modo de experimento já tinha sido removido, a execução foi automatizada por um script de shell que chama o programa uma vez para cada chave (Código 15), de modo que o índice é recriado a cada execução. As métricas de criação são as médias dessas 20 execuções; transferências e comparações de criação não variam entre elas, só o tempo.

```
i=0
while [ $i -lt 20 ]; do
  k=$(( 1 + i * (q - 1) / 19 ))
  ./pesquisa $m $q $s $k
  i=$((i+1))
done
```

Código 15: Escolha das 20 chaves do experimento (script de shell).

3.1 Limitações da bateria de testes

Algumas combinações não foram executadas por limite da máquina ou da própria implementação (Tabela 2). Os dois motivos principais foram a falta de espaço (de memória principal e de disco) e o tempo de execução, que em alguns casos passaria de horas ou até dias.

Combinação não executada	Motivo
Método 1, situações 2 e 3	O índice de páginas exige arquivo ordenado ascendentemente.
Métodos 3 e 4 com 2.000.000	Falta de espaço e tempo. As árvores guardam os registros completos em RAM: com 200.000 registros o pico foi de 1,0 a 1,9 GB, então com 2.000.000 seriam cerca de 10 a 19 GB, no limite ou acima dos 16 GB da máquina. Passando desse limite, o sistema usaria o disco como memória (*swap*) e a criação ficaria muito lenta. Além disso, cada arquivo de 2.000.000 de registros ocupa cerca de 10 GB, ou seja, 30 GB só para as três situações.
Método 2, situações 1 e 2, com 200.000 e 2.000.000	A árvore degenera em lista e a construção faz cerca de $n^2/2$ leituras. Com 20.000 levou cerca de 160 s; por extrapolação, 200.000 levaria cerca de 4,5 horas e 2.000.000 cerca de 18 dias.

Tabela 2: Combinações não executadas.

A função `limparCacheArquivo` usa `posix_fadvise`, que não existe no macOS. Por isso, até 200.000 registros os arquivos (no máximo 1 GB) podem ter sido lidos do cache do sistema operacional. Com 2.000.000 (cerca de 10 GB por arquivo) as leituras chegam de fato ao disco, o que explica o salto de tempo nesse tamanho.

4 Resultados

4.1 Método 1 – Acesso sequencial indexado

	Criação			Pesquisa (média)		
Registros	Transf.	Comp.	Tempo (ms)	Transf.	Comp.	Tempo (μs)
100	1	0	0,100	100	51,05	82,2
200	2	0	0,021	100	52,05	16,6
2.000	20	0	0,064	100	61,50	25,0
20.000	200	0	0,873	100	151,50	42,3
200.000	2.000	0	3,640	100	1.051,50	35,4
2.000.000	20.000	0	2.371,288	100	10.051,50	226,9

Tabela 3: Método 1, situação ascendente.

- **Criação – transferências**: exatamente $n/100$ (uma leitura por página), crescimento linear com fator pequeno.
- **Criação – comparações**: zero, pois o índice só copia a primeira chave de cada página.
- **Criação – tempo**: abaixo de 4 ms até 200.000 registros. Em 2.000.000 o arquivo tem cerca de 10 GB e não cabe no cache do sistema, então cada salto exige leitura real do disco e o tempo sobe para 2,4 s.
- **Pesquisa – transferências**: constante e igual a 100 em todos os tamanhos, porque a página inteira é trazida para a memória.
- **Pesquisa – comparações**: em média $(n/100)/2$ na tabela mais cerca de 50 na página, ou seja, crescem linearmente: de 51 (100 registros) para 10.051 (2.000.000). Uma pesquisa binária na tabela reduziria esse termo para $\log_2(n/100)$.
- **Pesquisa – tempo**: dominado pela leitura de uma página de cerca de 500 KB, por isso fica entre 17 e 82 μs até 200.000; em 2.000.000 sobe para 227 μs porque a página tem de vir do disco. O valor de 100 registros (82 μs), maior que o de 200, reflete a variação natural de medições na casa dos microssegundos.

4.2 Método 2 – Árvore binária de pesquisa em memória externa

		Criação			Pesquisa (média)		
Registros	Situação	Transf.	Comp.	Tempo (ms)	Transf.	Comp.	Tempo (μs)
100	Asc.	5.050	9.900	11,17	50,05	99,10	75,0
100	Desc.	5.050	4.950	6,25	50,95	100,90	46,5
100	Aleat.	811	975	1,63	8,70	16,40	13,2
200	Asc.	20.100	39.800	14,78	100,05	199,10	68,7
200	Desc.	20.100	19.900	14,68	100,95	200,90	69,5
200	Aleat.	1.793	2.294	2,64	9,25	17,50	17,4
2.000	Asc.	2.001.000	3.998.000	1.396,20	1.000,05	1.999,10	634,2
2.000	Desc.	2.001.000	1.999.000	1.414,97	1.000,95	2.000,90	637,4
2.000	Aleat.	26.070	37.152	33,18	12,80	24,60	56,2
20.000	Asc.	200.010.000	399.980.000	159.494,68	10.000,05	19.999,10	7.417,2
20.000	Desc.	200.010.000	199.990.000	163.856,85	10.000,95	20.000,90	7.509,4
20.000	Aleat.	374.285	506.246	475,72	17,60	34,20	77,6
200.000	Aleat.	4.757.935	6.828.679	5.641,92	21,00	41,00	32,4
2.000.000	Aleat.	53.193.586	76.939.985	817.895,59	27,40	53,80	4.760,3

Tabela 4: Método 2. Situações 1 e 2 com 200.000 e 2.000.000 não foram executadas (Tabela 2).

- **Criação – transferências**: nos arquivos ordenados são exatamente $n(n+1)/2$ (n leituras de dados mais $0+1+\cdots+(n-1)$ nodos visitados), crescimento quadrático: 5.050 para 100 registros

e 200 milhões para 20.000. No aleatório são cerca de n mais n vezes a profundidade média; com 2.000.000 dá cerca de 26 nodos lidos por inserção, crescimento $O(n \log n)$.

- **Criação – comparações**: no ascendente são $n(n-1)$, o dobro do descendente ($n(n-1)/2$), porque descer à direita exige dois testes (menor que e maior que) e descer à esquerda só um. No aleatório crescem como $n \log n$ (cerca de 38 por registro com 2.000.000).
- **Criação – tempo**: acompanha as transferências. Nos ordenados passa de 1,4 s (2.000) para cerca de 160 s (20.000), ou seja, 100 vezes mais para 10 vezes mais registros. No aleatório fica em 5,6 s com 200.000, mas chega a 818 s (13,6 min) com 2.000.000, porque o arquivo da árvore (cerca de 10 GB) não cabe na RAM e cada visita a um nodo vira um acesso aleatório ao disco.
- **Pesquisa – transferências**: nos ordenados, cerca de $n/2$ em média (as 20 chaves cobrem a lista de ponta a ponta); no aleatório, logarítmicas: de 8,7 (100) para 27,4 (2.000.000), perto de $1{,}39 \log_2 n$, valor esperado para uma árvore binária aleatória [4].
- **Pesquisa – comparações**: cerca de duas por nodo visitado (igualdade e menor que), logo o dobro das transferências.
- **Pesquisa – tempo**: cada nodo lido é um `fseek` mais um `fread` de cerca de 5 KB. No aleatório fica abaixo de 80 μs até 200.000 e sobe para 4,8 ms com 2.000.000 (leituras reais de disco); nos ordenados chega a 7,5 ms com apenas 20.000 registros.

4.3 Método 3 – Árvore B

		Criação			Pesquisa (média)	
Registros	Situação	Transf.	Comp.	Tempo (ms)	Comp.	Tempo (μs)
100	Asc.	100	5.148	0,163	10,40	0,25
100	Desc.	100	5.246	0,783	10,40	0,25
100	Aleat.	100	5.330	0,386	10,40	0,30
200	Asc.	200	20.298	0,237	13,00	0,10
200	Desc.	200	20.496	2,204	13,00	0,25
200	Aleat.	200	20.677	1,296	13,00	0,25
2.000	Asc.	2.000	315.442	3,405	20,00	0,55
2.000	Desc.	2.000	304.144	32,539	19,90	0,70
2.000	Aleat.	2.000	297.673	17,342	18,80	0,40
20.000	Asc.	20.000	5.038.905	36,578	26,40	1,80
20.000	Desc.	20.000	3.167.013	348,823	26,20	1,50
20.000	Aleat.	20.000	3.726.010	215,815	25,60	0,90
200.000	Asc.	200.000	61.680.903	411,788	33,00	4,50
200.000	Desc.	200.000	32.394.143	3.610,188	32,40	8,55
200.000	Aleat.	200.000	44.829.909	3.549,683	32,70	2,50

Tabela 5: Método 3 (ordem 100). Transferências na pesquisa: 0 em todas as linhas. O caso de 2.000.000 não foi executado (Tabela 2).

- **Criação – transferências**: exatamente n, pois o arquivo é lido uma única vez, sequencialmente.
- **Criação – comparações**: crescem mais que $n \log n$ porque, com páginas de até 200 registros, a descida na página e o deslocamento em `inserirNaPagina` são sequenciais. Por registro inserido são cerca de 52 (100 registros) e de 162 a 308 (200.000). O ascendente é o mais caro (61,7 milhões com 200.000), porque toda inserção vai para a página mais à direita e percorre todas as chaves dela; no descendente a posição é achada logo na primeira chave (32,4 milhões).
- **Criação – tempo**: aqui a ordem do arquivo pesa muito. Com 200.000 registros o ascendente leva 0,41 s, contra cerca de 3,6 s no descendente e no aleatório. No ascendente cada registro entra no fim da página e nada é deslocado; no descendente ele entra no início e `inserirNaPagina` precisa empurrar até 200 registros de 5 KB (cerca de 1 MB de cópia por inserção). Com a ordem 2 usada antes, os três casos ficavam em torno de 0,75 s.

- **Pesquisa – comparações**: logarítmicas e praticamente independentes da ordem do arquivo: de 10,4 (100 registros) para cerca de 33 (200.000). A árvore ficou bem mais baixa (3 níveis com 200.000), mas o total de comparações quase não mudou, porque a pesquisa binária dentro de cada página faz cerca de $\log_2$ do tamanho da página; no total continuam sendo cerca de $\log_2 n$ passos, cada um com duas comparações.

- **Pesquisa – tempo**: abaixo de 9 μs em todos os casos. Assim como antes, o aleatório pesquisa mais rápido que os ordenados nos tamanhos maiores, efeito do arranjo das páginas na memória e não do algoritmo. Valores abaixo de 1 μs estão no limite da resolução do relógio e servem só como ordem de grandeza.

- **Memória**: com 200.000 registros o pico do processo ficou entre 1,0 e 1,3 GB (antes, com ordem 2, chegava a 2,2 GB).

- **Por que não há teste com 2.000.000**: como a árvore guarda os registros completos na memória principal, ela precisaria de cerca de 10 a 13 GB de RAM. A máquina tem 16 GB, que também são usados pelo sistema operacional; o processo passaria a usar *swap* e a criação ficaria muito lenta. Também faltaria espaço em disco, já que cada arquivo de 2.000.000 de registros tem cerca de 10 GB.

4.4 Método 4 – Árvore B*

		Criação			Pesquisa (média)	
Registros	Situação	Transf.	Comp.	Tempo (ms)	Comp.	Tempo (μs)
100	Asc.	100	5.049	0,102	51,00	0,20
100	Desc.	100	5.148	0,578	51,00	0,00
100	Aleat.	100	5.234	0,352	51,00	0,10
200	Asc.	200	12.850	0,215	40,50	0,15
200	Desc.	200	12.900	1,568	40,50	0,35
200	Aleat.	200	12.770	0,809	39,50	0,15
2.000	Asc.	2.000	187.486	2,365	49,80	0,65
2.000	Desc.	2.000	153.138	16,920	46,45	0,65
2.000	Aleat.	2.000	163.136	9,516	47,75	0,20
20.000	Asc.	20.000	2.940.651	25,450	66,25	1,55
20.000	Desc.	20.000	1.589.845	179,386	61,55	2,60
20.000	Aleat.	20.000	2.155.194	118,890	73,10	0,80
200.000	Asc.	200.000	37.705.372	281,684	94,25	4,15
200.000	Desc.	200.000	15.996.230	1.855,532	95,85	3,05
200.000	Aleat.	200.000	24.516.899	2.178,756	100,95	1,70

Tabela 6: Método 4 (páginas de 100 itens). Transferências na pesquisa: 0 em todas as linhas. O caso de 2.000.000 não foi executado (Tabela 2).

- **Criação – transferências**: exatamente n, uma leitura sequencial do arquivo.

- **Criação – comparações**: menores que as da árvore B em todos os casos (37,7 milhões contra 61,7 milhões no ascendente com 200.000), porque as folhas e as páginas internas têm metade do tamanho das páginas da B. Como na B, o ascendente é o mais caro e o descendente o mais barato (16,0 milhões).

- **Criação – tempo**: de 35% a 50% menor que na árvore B (0,28 s contra 0,41 s no ascendente e cerca de 2 s contra 3,6 s nos demais, com 200.000). Os deslocamentos nas folhas movem no máximo 100 registros (cerca de 500 KB), e as divisões de páginas internas movem só chaves inteiras. A ordem do arquivo afeta o tempo pelo mesmo motivo da árvore B.

- **Pesquisa – comparações**: crescem devagar com n, mas são as maiores entre as árvores: 51 com 100 registros (uma única folha de 100, pesquisada sequencialmente) e cerca de 95 a 101 com 200.000. Como a pesquisa é sequencial dentro de cada página, cada nível custa em média metade

da página. O valor de 200 registros (40) é menor que o de 100 porque, depois da primeira divisão, as folhas ficam com 50 a 100 registros.

- **Pesquisa – tempo**: até 4,15 μs, em geral o menor entre os métodos apesar de mais comparações (a exceção é o descendente com 20.000, em que a B foi mais rápida). A explicação provável é a cache do processador: as chaves de uma página interna ficam juntas num vetor de `int`, enquanto na árvore B cada chave comparada está dentro de um registro de 5 KB.
- **Memória**: pico de 1,1 a 1,9 GB com 200.000 registros, o maior valor no ascendente. Nesse caso as folhas divididas ficam só com metade dos registros e não recebem mais inserções.
- **Por que não há teste com 2.000.000**: pelo mesmo motivo da árvore B. Os registros ficam nas folhas, em memória principal, e a árvore precisaria de cerca de 11 a 19 GB, acima do que a máquina tem livre. Com o *swap*, o tempo de criação cresceria muito, e os arquivos de teste ocupariam cerca de 30 GB de disco.

4.5 Análise comparativa

As Figuras 1 e 2 comparam os quatro métodos (método 1 no arquivo ascendente; métodos 2, 3 e 4 no arquivo aleatório) em escala logarítmica, e a Tabela 7 resume o comportamento de cada um.

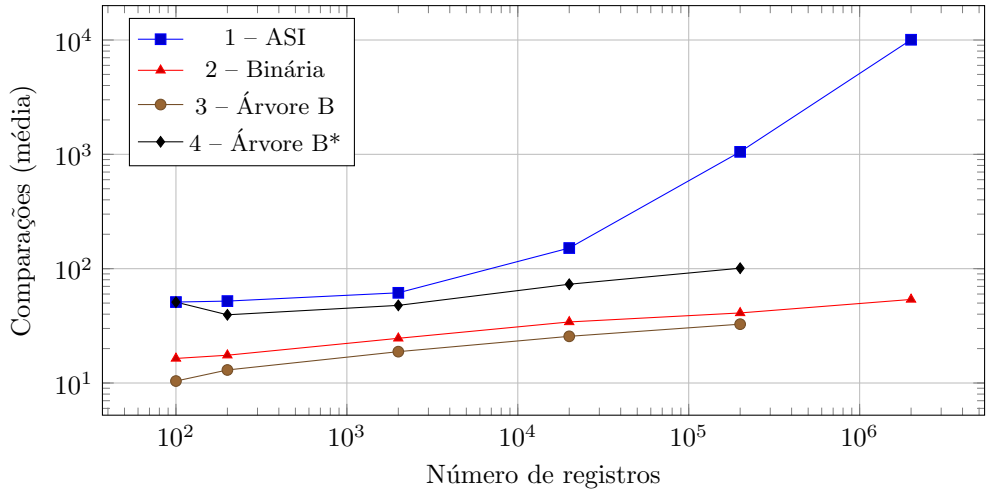


Figura 1: Comparações médias por pesquisa.

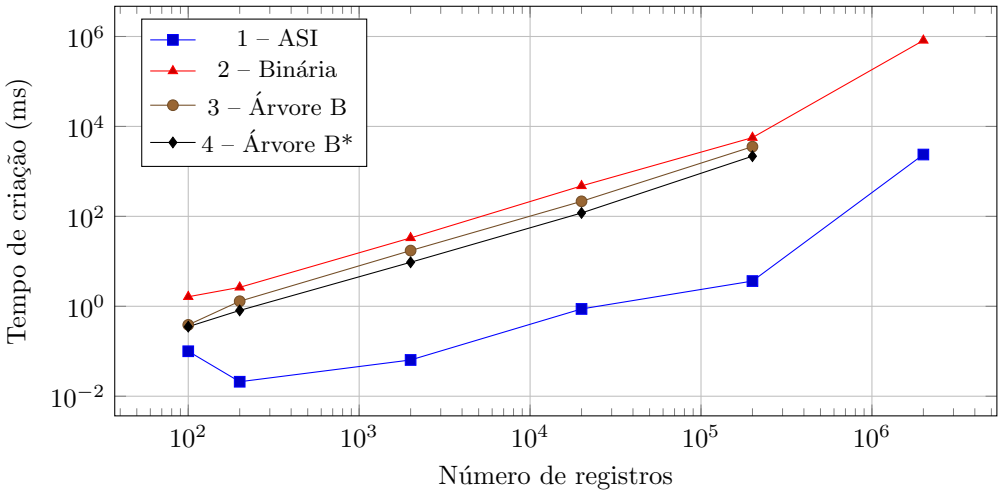


Figura 2: Tempo de criação do índice.

Método	Situações	Criação (transf.)	Pesquisa (transf.)	Pesquisa (comp.)	Memória interna
1 – ASI	Só ascendente	$n/100$	100 (constante)	Linear em n	Só a tabela de páginas
2 – Binária externa	Todas; viável só na aleatória	$n \log n$ (aleat.) / $n^2/2$ (ord.)	$\log n$ (aleat.) / $n/2$ (ord.)	$2\times$ as transf.	Nenhuma (tudo em disco)
3 – Árvore B	Todas	n	0	$\log n$ (a menor)	Árvore inteira (até 1,3 GB com 200.000)
4 – Árvore B*	Todas	n	0	Proporcional à altura vezes o tamanho da página	Árvore inteira (até 1,9 GB com 200.000)

Tabela 7: Resumo comparativo dos métodos.

- **Criação do índice**: o acesso sequencial indexado é imbatível (2.000 leituras e 3,6 ms com 200.000 registros) porque lê só um registro por página. Entre as árvores, a B* criou o índice de 35% a 50% mais rápido que a B, e a binária aleatória foi a mais lenta (5,6 s com 200.000) por ler do disco cada nodo visitado. Com páginas de 100 itens, porém, a criação das árvores B e B* no arquivo aleatório ficou bem mais lenta que antes (3,5 s e 2,2 s com 200.000), por causa do deslocamento de registros de 5 KB dentro de páginas grandes.

- **Pesquisa – transferências**: as árvores B e B* não fazem nenhuma, pois mantêm os registros em RAM; o método 1 faz sempre 100; a binária aleatória faz cerca de $1{,}39 \log_2 n$ (27 com 2.000.000).

- **Pesquisa – comparações**: a Figura 1 mostra a diferença de crescimento. O método 1 é linear (de 200.000 para 2.000.000 as comparações passam de 1.051 para 10.051, quase 10 vezes mais); as árvores crescem devagar. A B faz menos comparações (33 com 200.000) porque usa pesquisa binária dentro da página; a binária aleatória vem logo acima (41); a B*, com pesquisa sequencial em páginas de 100 chaves, é a que mais compara entre as árvores (cerca de 100), mas ainda bem menos que o método 1.

- **Pesquisa – tempo**: B* (até 4,2 μs) < B (até 8,6 μs) < método 1 (17 a 227 μs) < binária aleatória (13 μs a 4,8 ms). A binária com arquivo ordenado é a pior de todas (7,5 ms com só 20.000 registros). A B* é, em geral, a mais rápida mesmo comparando mais, provavelmente porque percorre vetores contíguos de `int`, enquanto a B compara chaves espalhadas em registros de 5 KB.

- **Ordem do arquivo**: não altera a pesquisa nas árvores B e B*, que se mantêm balanceadas, e altera apenas o custo de criação: o ascendente é o mais caro em comparações, mas o mais rápido em tempo, porque não exige deslocar registros dentro das páginas. Na árvore binária, arquivo ordenado transforma o custo logarítmico em linear na pesquisa e em quadrático na criação.

- **Escala**: com 2.000.000 de registros, os métodos que dependem do disco ficaram muito mais lentos (criação de 2,4 s no método 1 e de 818 s na binária), e as árvores B e B* não foram executadas por causa do uso de memória (Tabela 2).

5 Considerações Finais

As árvores B* e B foram os métodos mais eficientes na pesquisa (a B* com o menor tempo, a B com o menor número de comparações); o acesso sequencial indexado teve o índice mais barato; e a árvore binária só é viável com arquivo aleatório, situação em que foi o único método testado com 2.000.000 de registros guardando os dados somente em disco.

As principais dificuldades encontradas na implementação foram:

- **Árvore B***: foi o método mais trabalhoso. A divisão de páginas internas precisou ser refeita (juntar as chaves num vetor temporário antes de dividir), e os nomes de funções e tipos entraram em conflito com os da árvore B no mesmo programa e precisaram ser renomeados.
- **Uso de memória**: como as páginas das árvores B e B* guardam registros de 5 KB, o consumo de RAM cresce muito rápido. Na B* as páginas internas passaram a ser alocadas só com o tamanho da parte interna da `union`; mesmo assim não foi possível testar 2.000.000 de registros nas duas árvores. Guardar nas páginas apenas a chave e a posição do registro no arquivo resolveria isso, ao custo de uma transferência por pesquisa.
- **Árvore binária em disco**: manter os ponteiros como posições no arquivo e regravar o pai a cada inserção exigiu cuidado com `fseek`; além disso, a degeneração com arquivos ordenados tornou inviáveis os testes grandes nessas situações.
- **Medição justa**: foi preciso separar claramente o que entra em cada medição: as impressões ficam fora, e toda comparação entre chaves passa pela macro `COMP`, para que a contagem seja uniforme entre os métodos.
- **Arquivos grandes**: um arquivo de 2.000.000 de registros ocupa cerca de 10 GB (mais 10 GB do arquivo da árvore binária), o que exigiu apagar os arquivos entre os testes.

Como melhorias futuras, destacam-se o uso de pesquisa binária na tabela de índice do método 1 e dentro das páginas da árvore B* (o que tornaria as comparações logarítmicas nos dois casos) e o armazenamento apenas de chave e posição nas páginas das árvores B e B*, permitindo testá-las com 2.000.000 de registros.

Referências

[1] Rudolf Bayer and Edward M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

[2] Douglas Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition, 2009.

[4] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition, 1998.

[5] Nivio Ziviani. *Projeto de Algoritmos com Implementações em Pascal e C*. Cengage Learning, São Paulo, 3 edition, 2011.